

Lesson Activities — 2.3 Algorithms

In-lesson classroom activities for OCR A Level Computer Science H446, Component 02, Section 2.3.1 — human algorithm demos, Parson's problems, bug hunts and prediction tasks, not exam questions. All traces and complexities verified.

2.3.1 Big O & searching

Guess My Number in 7 (unplugged demo · 12 min · whole class)

Resources: Board; a hidden number 1-100 written on a folded slip; tally chart for guesses; calculator (for the log finale).

How it runs: 1. Announce: "I'm thinking of a number from 1 to 100. I will only answer *higher*, *lower* or *correct*. I bet the class can always get it in **7 guesses or fewer** — if you use the right algorithm." 2. Round 1 — let students guess freely (they'll often scatter-gun). Tally the guesses. 3. Round 2 — impose the algorithm: guess the **middle** of the remaining range every time, using $\text{mid} = (\text{low} + \text{high}) \text{ DIV } 2$. Track **low**, **high**, **mid** in three board columns. Play with the hidden number **73**. 4. Round 3 — adversarial: you secretly *move* the number to whichever half keeps the game alive longest (worst case). Class still wins by guess 7. Demonstrate with **100** as the target. 5. The reveal: each guess **halves** the remaining range: $100 \rightarrow 50 \rightarrow 25 \rightarrow 12 \rightarrow 6 \rightarrow 3 \rightarrow 1$. Ask: "how many halvings does 100 survive?" Board the maths: $2^6 = 64 < 100$ but $2^7 = 128 \geq 100$, so 7 guesses always suffice — and $\log_2(100) \approx 6.64 < 7$. This is **binary search**, and this is what **$O(\log n)$** means: doubling the range costs only *one* more guess.

Answers / expected outcomes: - Target 73 trace (low, high $\rightarrow$ mid): 1,100 $\rightarrow$ **50** (higher); 51,100 $\rightarrow$ **75** (lower); 51,74 $\rightarrow$ **62** (higher); 63,74 $\rightarrow$ **68** (higher); 69,74 $\rightarrow$ **71** (higher); 72,74 $\rightarrow$ **73** ✓ — found in **6 guesses**. - Worst-case target 100: guesses 50, 75, 88, 94, 97, 99, 100 — exactly **7 guesses**. - Punchlines to land: 1,000,000 numbers need only 20 guesses ($2^{20} = 1,048,576$); doubling n adds **one** step; and the precondition — the "list" 1-100 is **sorted**, which is the only reason halving works.

Stretch: How many guesses for 1 to 1,000,000,000? (30, since $2^{30} \approx 1.07 \times 10^9$.) Why does the same trick fail on an unsorted pile of numbered cards? (Discarding half is only safe if order tells you which half cannot contain the target.)

Big O Graph-Sketch Race (race · 12 min · pairs, mini-whiteboards)

Resources: Mini-whiteboards or graph paper per pair; six expression cards per pair; visible timer.

How it runs: 1. **Heat 1 (4 min):** pairs sketch all six growth curves on ONE set of axes ($x = n$, $y =$ operations), correctly ordered and labelled: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$. First pair to hold up a fully correct board wins; spot-check by asking for the value of each at $n = 8$. 2. **Heat 2 (3 min):** hold up expression cards one at a time; pairs write the Big O class: $4n + 7 \cdot 3n^2 + 5n + 17 \cdot n(n-1)/2 \cdot 500 \cdot 2n \log n + 3n \cdot 2^n + n^3$. 3. **Heat 3 (3 min):** "algorithm snap" — call out algorithms (linear search, binary search, bubble sort worst, merge sort, quick sort average, quick sort worst); pairs point at the matching curve on their sketch. 4. Debrief the rule that decided every heat: keep the **dominant term**, drop constants and lower-order terms; Big O is **worst-case** growth of time or space against n .

Answers / expected outcomes: - Curve order check at $n = 8$: $O(1) = 1 \cdot O(\log n) = 3 \cdot O(n) = 8 \cdot O(n \log n) = 24 \cdot O(n^2) = 64 \cdot O(2^n) = 256$. Curves must never cross in the wrong order for large n . - Heat 2: $4n + 7 \rightarrow O(n)$; $3n^2 + 5n + 17 \rightarrow O(n^2)$; $n(n-1)/2 \rightarrow O(n^2)$; $500 \rightarrow O(1)$; $2n \log n + 3n \rightarrow O(n \log n)$; $2^n + n^3 \rightarrow O(2^n)$. - Heat 3: linear search $\rightarrow O(n)$; binary search $\rightarrow O(\log n)$; bubble worst $\rightarrow O(n^2)$; merge sort $\rightarrow O(n \log n)$; quick average $\rightarrow O(n \log n)$; quick worst $\rightarrow O(n^2)$.

Stretch: Where's the tractable/intractable border on the sketch? (Polynomial or better = tractable; $O(2^n)$ = intractable.) Trick card: $n + 2^{n^0}$ (i.e. $n + \text{constant}$) — still $O(n)$; and "an algorithm does 3 nested loops of n but we only ever run it on $n = 10$ — does Big O care?" (No: Big O describes growth, not one run.)

Parson's Problem: Rebuild Binary Search (Parson's problem · 12 min · pairs)

Resources: Envelope per pair with 12 shuffled code strips (print and cut).

How it runs: 1. Brief: "Rebuild a working binary search **function** that returns the index of `target` in sorted array `arr`, or `-1` if absent. Indentation is part of the answer." 2. Strips (shuffled in the envelope): - **A** endfunction - **B** high = arr.length - 1 - **C** if arr[mid] == target then - **D** return -1 - **E** function binarySearch(arr, target) - **F** mid = (low + high) DIV 2 - **G** low = mid + 1 - **H** while low <= high - **I** elseif arr[mid] < target then - **J** low = 0 - **K** return mid - **L** else / high = mid - 1 / endif / endwhile (one strip, 4 lines) 3. Gate to finish: pairs must trace their assembled code on `arr = [3, 7, 11, 18, 25, 42, 59]`, `target = 42`, writing every `low`, `high`, `mid` value, before you sign it off. 4. Debrief traps: why `while low <= high` and not `<`? (A one-element range `low == high` must still be checked.) Why `mid + 1 / mid - 1`, not `mid`? (Otherwise the range never shrinks — infinite loop.)

Answers / expected outcomes: - Correct order: **E, J, B, H, F, C, K, I, G, L, D, A** — i.e.: `function binarySearch(arr, target) low = 0 high = arr.length - 1 while low <= high mid = (low + high) DIV 2 if arr[mid] == target then return mid elseif arr[mid] < target then low = mid + 1 else high = mid - 1 endif endwhile return -1 endfunction` - Trace for target 42: `low=0, high=6, mid=3` $\rightarrow$ `arr[3]=18 < 42` $\rightarrow$ `low=4`; `low=4, high=6, mid=5` $\rightarrow$ `arr[5]=42` $\rightarrow$ **return 5**. Two iterations. - Trace for target 10 (stretch of the trace): `mid=3` (`18 > 10` $\rightarrow$ `high=2`); `mid=1` (`7 < 10` $\rightarrow$ `low=2`); `mid=2` (`11 > 10` $\rightarrow$ `high=1`); `low > high` $\rightarrow$ **return -1**.

Stretch: Convert it to a recursive version (parameters `low`, `high`; base case `low > high`). Which uses more memory and why? (Recursive: $O(\log n)$ stack frames vs $O(1)$ iterative.)

Odd One Out: Complexity Edition (odd one out · 8 min · small groups)

Resources: Three slides (or printed cards), one set of four items each; mini-whiteboards.

How it runs: 1. Show a set of four; groups pick the odd one out **and must state the rule that makes it odd**. Award credit for any defensible rule — then reveal the intended one. 2. Sets: - **Set A:** accessing `array[5]` · pushing onto a stack · linear search · dequeuing from a queue - **Set B:** binary search · merge sort · quick sort (average) · bubble sort - **Set C:** naïve recursive Fibonacci · brute-force Travelling Salesman · binary search · brute-force password cracking 3. For each set, a different group must propose a *second*, different but defensible odd-one-out to prove the point that classification depends on the property you choose.

Answers / expected outcomes: - **Set A:** linear search — $O(n)$; the other three are $O(1)$. - **Set B (intended):** bubble sort — $O(n^2)$; the others involve $O(\log n)/O(n \log n)$ "halving" behaviour. (Also defensible: binary search — the only *search*; merge sort — the only one with $O(n)$ extra space.) - **Set C:**

binary search — $O(\log n)$, tractable; the others are exponential $O(2^n)$ -style, intractable brute force. - Insist every answer is *justified* with a complexity — " $O(n^2)$ because..." is the habit being trained.

Stretch: Groups build their own set of four where **every** item is a defensible odd one out, and challenge another group to find all four justifications.

The Great Flip-Off: Linear vs Binary (human demo · 12 min · whole class)

Resources: 15 A4 cards, each with a value on the front, held **face down** in sorted order by 15 students (or blu-tacked along the board): 2, 5, 9, 12, 16, 21, 27, 33, 38, 44, 50, 57, 63, 71, 80. Two searcher students; two tally counters.

How it runs: 1. Line up the 15 card-holders (positions 0-14 labelled on the floor). Cards stay face down — a "flip" is one comparison, and flips are the only thing we count. 2. Announce the target: **57**. Searcher 1 runs **linear search**: flip position 0, 1, 2, ... until found. Counter tallies flips. 3. Reset (cards face down). Searcher 2 runs **binary search** aloud: "low 0, high 14, mid $(0+14) \text{ DIV } 2 = 7$ — flip position 7." Continue halving. Counter tallies flips. 4. Compare tallies on the board. Then ask: "when would linear search win?" (Target 2 — linear finds it in 1 flip; binary still takes several.) Run that round to prove best case $O(1)$ vs the *typical* behaviour. 5. Close with the two conditions: binary needed the line **sorted**; linear didn't care.

Answers / expected outcomes: - Target 57: linear search = **12 flips** (positions 0-11). Binary search: mid 7 → $33 < 57$ → low = 8; mid $(8+14) \text{ DIV } 2 = 11$ → $57 \checkmark$ — **2 flips**. - Target 2: linear = **1 flip**; binary = mid 7 (33, go left), mid 3 (12, go left), mid 1 (5, go left), mid 0 (2 ✓) = **4 flips** — which is also binary's worst case for 15 items, since 15 halves to $7 \rightarrow 3 \rightarrow 1$ ($2^4 - 1 = 15$). - Board the general result: linear $O(n)$ worst/average, $O(1)$ best; binary $O(\log n)$ worst/average, $O(1)$ best; binary requires sorted data.

Stretch: How many students would we need before binary's worst case reaches 5 flips? ($31 = 2^5 - 1$.) If the cards were *unsorted*, what would binary search return for 57 — and why is "sometimes the right answer" worse than "never"? (Silent wrong answers are the most dangerous bug.)

2.3.1 Sorting

Human Bubble Sort (human demo · 15 min · whole class)

Resources: 5 A4 number cards: **5, 1, 4, 2, 8**; 5 students in a line facing the class; a "comparison judge" student with two flags (SWAP / NO SWAP); tally board for comparisons and swaps; a chair at the end as the "sorted zone" marker.

How it runs: 1. Five students hold the cards in order **5, 1, 4, 2, 8** (audience's left to right = index 0 to 4). Rule: the judge walks along adjacent pairs; if the left card is **bigger**, the two students physically swap places. 2. Run **Pass 1** pair by pair, class calling SWAP/NO SWAP before the judge rules. After the pass, the rightmost student sits on the "sorted" chair line — largest has bubbled to the end, so the next pass ignores them. 3. Run **Pass 2** over the remaining 4 (3 comparisons), then **Pass 3** over 3 (2 comparisons). Keep a swap-flag card: if a whole pass has zero swaps, STOP. 4. Tally totals and ask: "why is this $O(n^2)$?" (Up to $n-1$ passes, each with up to $n-1$ comparisons — nested loops.) "When is it $O(n)$?" (Already-sorted input, one pass, zero swaps, flag stops it — **only** because we used the flag.)

Answers / expected outcomes: - **Pass 1** (4 comparisons, 3 swaps): (5,1) → swap → 1 5 4 2 8; (5,4) → swap → 1 4 5 2 8; (5,2) → swap → 1 4 2 5 8; (5,8) → no swap. Result: **1 4 2 5 8** — 8 is fixed. - **Pass**

2 (3 comparisons, 1 swap): (1,4) no; (4,2) → swap → 1 2 4 5 8; (4,5) no. Result: **1 2 4 5 8** — 5 fixed. - **Pass 3** (2 comparisons, 0 swaps): (1,2) no; (2,4) no → swap flag clean → STOP. - Totals: **3 passes, 9 comparisons, 4 swaps**, final order 1 2 4 5 8. After pass *k*, the *k* largest values are guaranteed in their final positions.

Stretch: Re-run starting from **8 5 4 2 1** (near-worst case) and predict the swap count first (it's 9: every pair is inverted except none involving... have the class verify by counting inversions). Then re-run on **1 2 4 5 8** to show the flag giving one $O(n)$ pass.

Bug Hunt: Insertion Sort (spot-and-fix-the-bug · 12 min · pairs)

Resources: Printed buggy code (below) per pair; the test list [5, 3, 8, 1]; mini-whiteboards for the trace.

```
01 procedure insertionSort(arr)
02   for i = 1 to arr.length - 1
03     current = arr[i]
04     j = i - 1
05     while j >= 0 AND arr[j] < current
06       arr[j + 1] = arr[j]
07     endwhile
08     arr[j] = current
09   next i
10 endprocedure
```

How it runs: 1. Tell pairs there are **exactly three bugs**: one makes it sort the wrong way, one makes it hang forever, one puts the value in the wrong slot. Find, name and fix all three. 2. Rule: no fix is accepted without evidence — pairs must show the line, say what goes wrong at run time, and give the corrected line. 3. Once fixed, pairs hand-trace [5, 3, 8, 1] on whiteboards, writing the array after each value of *i*. 4. Debrief: why does insertion sort suit *nearly sorted* data? (The while loop exits immediately when no shifting is needed — best case $O(n)$.)

Answers / expected outcomes: - **Bug 1 (line 05):** `arr[j] < current` shifts smaller elements, sorting **descending**. Fix: `arr[j] > current` for ascending. - **Bug 2 (lines 05-07):** `j` is never decremented — infinite loop whenever a shift happens. Fix: add `j = j - 1` inside the while. - **Bug 3 (line 08):** `arr[j] = current` writes one slot too far left (and overwrites data). Fix: `arr[j + 1] = current` — after the loop, `j` sits one *before* the insertion point. - Corrected trace of [5, 3, 8, 1]: *i*=1 (current 3): shift 5 → [3, 5, 8, 1]; *i*=2 (current 8): no shifts → [3, 5, 8, 1]; *i*=3 (current 1): shift 8, 5, 3 → [1, 3, 5, 8]. - Complexity check: worst $O(n^2)$, best $O(n)$, stable, in-place $O(1)$ space.

Stretch: What single input makes the *fixed* code do the most shifting? (Reverse-sorted, e.g. [8,5,3,1].) Adapt the code to sort an array of records by one field — does stability now matter? (Yes: equal keys keep their original relative order, so a previous sort by another field survives.)

Explain Merge Sort to the Duck (rubber-duck verbalisation · 12 min · pairs)

Resources: One rubber duck (or any toy) per pair; the list [6, 5, 3, 1, 8, 7, 2, 4] on 8 small cards per pair; a "duck script" prompt card: "*The duck knows nothing. It interrupts whenever you say 'just' or 'obviously'.*"

How it runs: 1. Partner A physically performs merge sort with the cards while explaining every move **to the duck** — splitting rows of cards apart, then merging pairs of sorted rows by repeatedly comparing the two front cards. Partner B holds the duck, listens for gaps, and interrupts (as the duck) whenever a step is asserted but not explained. 2. Swap roles at the merge stage: Partner B must explain *why* merging two sorted lists only ever compares the two front cards. 3. Finish with the duck's three exam questions (on the prompt card): "Why is this divide and conquer?" "Why $O(n \log n)$?" "What does it cost you?" — both partners agree an answer aloud.

Answers / expected outcomes: - Correct physical trace: split $\rightarrow$ [6,5,3,1] [8,7,2,4] $\rightarrow$ [6,5] [3,1] [8,7] [2,4] $\rightarrow$ singles $\rightarrow$ merge pairs $\rightarrow$ [5,6] [1,3] [7,8] [2,4] $\rightarrow$ merge $\rightarrow$ [1,3,5,6] [2,4,7,8] $\rightarrow$ final merge $\rightarrow$ [1,2,3,4,5,6,7,8]. - Duck answers: divide and conquer = repeatedly split into smaller instances of the same problem, solve (single cards are trivially sorted), then **combine** by merging. $O(n \log n) = \log_2 n$ levels of splitting/merging (here $\log_2 8 = 3$ levels) $\times O(n)$ work merging per level. The cost: **$O(n)$ extra space** for the merge lists — the classic time-vs-space trade-off against bubble/insertion. - Listen for the key merge invariant: both sublists are sorted, so the smallest unplaced card is always one of the two front cards.

Stretch: The duck asks: "what if the list length isn't a power of 2, say 7 cards?" (Splits are uneven — 4 and 3 — but the algorithm is unchanged and still $O(n \log n)$.) And: "is merge sort stable?" (Yes — when front cards tie, take from the left list.)

Pivot! The Human Quick Sort Partition (human demo · 15 min · whole class)

Resources: 7 A4 number cards: **7, 2, 9, 4, 8, 5, 1**; 7 students; three floor zones taped LEFT / PIVOT / RIGHT; a crown (or hat) for the pivot.

How it runs: 1. Seven students line up holding **7 2 9 4 8 5 1**. Crown the **first** student (7) as the pivot; they stand in the PIVOT zone. 2. Each remaining student, one at a time in order, compares themselves with the pivot and walks to LEFT (smaller) or RIGHT (larger), keeping their arrival order within the zone. 3. Freeze. Ask the killer question: "Is anyone *definitely* in their final sorted position?" (Only the pivot — everyone smaller is left of it, everyone larger is right of it.) 4. Recurse: crown a new pivot in the LEFT zone (first arrival) and partition it; likewise RIGHT. Continue until every zone has ≤ 1 person; then read the line left to right. 5. Debrief: why $O(n \log n)$ *on average* (each partition level does $\sim n$ comparisons, and halving gives $\sim \log n$ levels), and what goes wrong if the input is already sorted and we keep picking the first element? (Every partition puts everyone on one side — n levels $\rightarrow O(n^2)$ worst case.)

Answers / expected outcomes: - First partition, pivot 7: LEFT = **2, 4, 5, 1** (arrival order), PIVOT = 7, RIGHT = **9, 8**. Pivot 7 lands in final position 5 of 7 (index 4). - LEFT recursion, pivot 2: LEFT = [1], RIGHT = [4, 5]; then pivot 4: RIGHT = [5]. LEFT zone resolves to 1 2 4 5. - RIGHT recursion, pivot 9: LEFT = [8]. Resolves to 8 9. - Final line: **1 2 4 5 7 8 9**. Comparisons in the first partition: 6 (everyone vs pivot). - To demo the worst case, re-run with **1 2 4 5 7 8 9** and first-element pivots: every RIGHT zone gets everyone; partitions of size 6, 5, 4, 3, 2, 1 $\rightarrow$ 21 comparisons and n levels of recursion.

Stretch: Re-run the sorted-input case choosing the **middle** person as pivot — near-perfect halving returns. Discuss real fixes: random pivot / median-of-three. Is this human quick sort stable? (No — walking to zones can reorder equal values; quick sort is not stable.)

Always, Sometimes, Never: Sorting Claims (always-sometimes-never · 10 min · whole class)

Resources: ALWAYS / SOMETIMES / NEVER corner posters; statement slides; mini-whiteboards for justifications.

How it runs: 1. Reveal statements one at a time; students commit to a corner, then defend before adjudication. Every verdict must cite a case or a complexity. 2. Statements: - **S1** "After one full pass of bubble sort, the largest value is in its final position." - **S2** "Bubble sort's best case is $O(n)$." - **S3** "Quick sort sorts in $O(n \log n)$ time." - **S4** "Merge sort (as taught for H446) needs extra memory proportional to n ." - **S5** "A stable sort keeps equal items in their original relative order." - **S6** "Insertion sort is slower than merge sort." - **S7** "Sorting a list first makes finding an item in it faster."

Answers / expected outcomes: - **S1 Always** — each pass bubbles the largest remaining value to the end; that's the loop invariant. - **S2 Sometimes** — $O(n)$ on already-sorted data **only with a swap flag**; without the flag it does all passes $\rightarrow O(n^2)$. - **S3 Sometimes** — $O(n \log n)$ average/best, but **$O(n^2)$** worst case (poor pivot, e.g. first element on sorted data). - **S4 Always** — the merge step builds new lists $\rightarrow O(n)$ auxiliary space. - **S5 Always** — that is the definition of stable. - **S6 Sometimes** — asymptotically $O(n^2)$ vs $O(n \log n)$, but on small or nearly-sorted lists insertion sort's $O(n)$ best case wins. - **S7 Sometimes** — it enables $O(\log n)$ binary search, but the $O(n \log n)$ sort only pays off if you search often enough; one single lookup is faster with linear search on the unsorted list.

Stretch: Pairs write one new sorting ASN statement that would split the class, and must know their own verdict + justification before offering it.

2.3.1 Data-structure algorithms & traversals

Walk the Tree (human demo · 15 min · whole class)

Resources: The BST taped on the floor with masking tape (nodes = paper plates, edges = tape): root **50**; left child **30** (children **20**, **40**); right child **70** (children **60**, **80**). One walker; class with mini-whiteboards; three signs: PRE / IN / POST.

How it runs: 1. Teach the walking rule: the walker starts at the root and walks **anticlockwise around the outside of the whole tree**, keeping their left hand towards it, visiting left subtrees before right, and returning through each node. 2. Every node is touched up to three times: on the way **down** (first touch), **between** its left and right subtrees (middle touch), and on the way back **up** (last touch). Do one slow lap with no shouting so the class sees the three touch types. 3. **Pre-order lap:** walker shouts each node's value on its **first** touch; class writes the sequence. 4. **In-order lap:** shout on the **middle** touch (for a leaf, its single touch). Ask what the class notices about the output. 5. **Post-order lap:** shout on the **last** touch. 6. **BFS finale:** no walking around — a "queue" of students at the side: enqueue 50; repeatedly dequeue-and-shout a node then enqueue its children left-to-right. Contrast: the three walks were depth-first (one route, a *stack* of the path walked); BFS needed a **queue**.

Answers / expected outcomes: - Pre-order (Node, Left, Right): **50, 30, 20, 40, 70, 60, 80** — used to copy a tree. - In-order (Left, Node, Right): **20, 30, 40, 50, 60, 70, 80** — **ascending order**, because it's a BST; this is the observation to force out in step 4. - Post-order (Left, Right, Node): **20, 40, 30, 60, 80, 70, 50** — children before parents (delete a tree; evaluate postfix/RPN). - BFS (level order): **50, 30, 70, 20, 40, 60, 80**. - Mnemonic: PRE/IN/POST names where the **Node** goes — first, middle, last.

Stretch: Insert **65** (walker decides the path: 50 → right, 70 → left, 60 → right; 65 becomes 60's right child) and re-run in-order to confirm it still comes out ascending. Then ask which traversal, run on the floor tree, would let you rebuild an identical tree by re-inserting values into an empty BST in that order (pre-order).

What Prints Next? Stacks & Queues (live-coding prediction · 10 min · individual, mini-whiteboards)

Resources: Mini-whiteboards; the script below revealed one numbered line at a time (slides or IDE).

```
01 stack = new Stack()           // LIFO
02 queue = new Queue()           // FIFO
03 stack.push(3)
04 stack.push(8)
05 stack.push(5)
06 print(stack.pop())
07 print(stack.pop())
08 stack.push(1)
09 print(stack.pop())
10 queue.enqueue("red")
11 queue.enqueue("blue")
12 queue.enqueue("green")
13 print(queue.dequeue())
14 queue.enqueue("gold")
15 print(queue.dequeue())
16 print(queue.dequeue())
```

How it runs: 1. Reveal lines 01-05, then stop before every `print`: boards up — "what prints next?" Reveal, then continue. 2. After line 09 ask for the full current stack contents (bottom → top); after line 16 the full queue. 3. Sting question at the end: "which of these 14 operations was *not* O(1)?" (None — that's the point: push/pop/enqueue/dequeue are all O(1); nothing shuffles along.) 4. Close by boarding the pairing students must never mix up: **stack** → **LIFO** → **DFS**; **queue** → **FIFO** → **BFS**.

Answers / expected outcomes: - Line 06 → **5**; line 07 → **8**; line 09 → **1** (the just-pushed 1, not 3 — the classic error). - After line 09 the stack holds **[3]** (bottom → top). - Line 13 → **red**; line 15 → **blue** (gold joined the **rear** — a second classic error is answering gold); line 16 → **green**. - Final state: stack [3], queue [gold]. - Full printed output in order: **5, 8, 1, red, blue, green**.

Stretch: Re-run lines 10-16 as a **circular queue** of capacity 4 with front/rear pointers on the board — show the rear pointer wrapping with `MOD 4` when gold is enqueued after red leaves, and why wrapping lets us reuse freed space.

Driver & Navigator: Recursive Traversal (pair programming · 20 min · pairs)

Resources: One computer per pair; starter file with a `Node` class/record (`value`, `left`, `right`) and the seven-node tree from *Walk the Tree* pre-built; visible 5-minute swap timer.

How it runs: 1. Assign roles: **driver** types, **navigator** dictates strategy and hunts bugs — navigators may not touch the keyboard; drivers may not write a line the navigator hasn't agreed. Swap roles every 5 minutes on the timer. 2. Sprint 1: write recursive `inOrder(node)` and run it on the tree. The expected

output is on the board *before* they start (so testing is honest): 20 30 40 50 60 70 80 . 3. Sprint 2 (roles swapped): write `postOrder(node)` by moving **one line** — navigators must articulate which line and why before the driver touches it. 4. Sprint 3 (swap again): add a counter that records the **maximum recursion depth** reached, and check it against the tree's height. 5. Debrief: where's the base case in this recursion? (`node == null` — the `if` guard; every call pushes a frame, so `depth = tree height`.)

Answers / expected outcomes: - Target code shape (OCR-style): `procedure inOrder(node) if node != null then inOrder(node.left) print(node.value) inOrder(node.right) endif endprocedure` - Sprint 1 output: **20 30 40 50 60 70 80** (in-order on a BST = ascending). - Sprint 2: move `print(node.value)` **after** the two recursive calls → post-order output **20 40 30 60 80 70 50**. - Sprint 3: maximum depth for the balanced 7-node tree = **3** levels of node calls (50 → 30 → 20; a `null`-check guard adds one more if they count the null call — accept either if justified). Traversal visits every node once: **O(n)**.

Stretch: Code-golf finale: shortest working `preOrder` wins (whitespace ignored, must still be readable enough for the navigator to defend every token). Then: what tree shape makes the recursion depth O(n)? (A degenerate/linear BST — insert 1, 2, 3, ... in order.)

The Human Linked List (unplugged demo · 12 min · whole class)

Resources: 5 students; A4 value cards **12, 99, 7** plus spares (**50, 31**); one foam "HEAD" arrow (or a pointing student); students point with their right arm as their `next` pointer; a NULL sign on the wall for the last node to point at.

How it runs: 1. Build the list: HEAD → 12 → 99 → 7 → NULL. Each student holds a value and physically points at the next; 7 points at the NULL sign. 2. **Traverse:** a "CPU" student starts at HEAD and walks pointer to pointer, reading values aloud. Count the steps. (You can only enter the list at HEAD — no jumping to the middle.) 3. **Insert 50 after 12:** student 50 stands up *anywhere in the room*, points at whoever 12 currently points at (99); then 12 re-points at 50. Count the pointer changes: two arm movements, **no one shuffled position** — contrast loudly with an array insert where everyone right of the gap must step sideways. 4. **Delete 99:** 50 re-points at 7. One arm movement. Ask 99 (still standing, pointed-at by nobody) what they are now. (Garbage — unreachable.) 5. **The gotcha:** try to insert "31 after 99" — the CPU must first *find* 99 by walking from HEAD... but 99 is gone. Walk the whole list to discover this. Finding = O(n) even though re-pointing = O(1).

Answers / expected outcomes: - After step 3 the list reads HEAD → 12 → 50 → 99 → 7; after step 4: HEAD → **12** → **50** → **7** → NULL. - Costs to board: traverse/search **O(n)**; insert/delete **given the node = O(1)** (re-pointing only); array equivalents: access by index O(1) but insert/delete O(n) shifting. - Students should articulate the trade-off pair: arrays win at random access; linked lists win at insertion/deletion once positioned.

Stretch: What breaks if you do step 3's pointer changes in the wrong order (12 points at 50 *first*)? (50 has nothing to point at — 99 and the rest of the list are lost; order of pointer updates matters.) Make it a doubly linked list (left arms = `prev`): how many arm movements does delete now take? (Four: two neighbours × two directions.)

Stack Kid vs Queue Conga: Human DFS & BFS (human demo · 15 min · whole class)

Resources: 5 students as graph nodes A-E standing at taped floor spots, holding rope/string edges: **A-B, A-C, B-D, C-D, D-E**; one explorer with a tray of cups (the stack); one "conga line" area (the queue);

sticky dots to mark visited nodes.

How it runs: 1. Board the adjacency list first — tie-break rule is **alphabetical**: A: B,C · B: A,D · C: A,D · D: B,C,E · E: D. 2. **DFS from A (stack/recursion)**: the explorer walks the ropes, always taking the alphabetically-first unvisited neighbour, stacking a cup for each forward step (push) and removing one each time they must walk back (pop). Class records the visit order as dots go on. 3. **BFS from A (queue)**: node A joins the conga line. Repeatedly: the front person leaves the line (dequeue), gets their visited dot, and taps all their un-dotted, not-yet-queued neighbours to join the **back** of the line (enqueue), alphabetically. 4. Compare the two visit orders on the board and ask why they differ at the third node. 5. Land the rule: **DFS = stack (or recursion — same thing, the call stack), BFS = queue**. Ask which one found E via the *fewest-edges* route (BFS — level by level, so it gives shortest paths in an **unweighted** graph).

Answers / expected outcomes: - DFS visit order: **A, B, D, C, E** (A → B → D; at D try C — unvisited, visit C; C's neighbours A, D both visited, so pop back to D; visit E). Maximum cups stacked: 4 (path A-B-D-C). - BFS visit order: **A, B, C, D, E** (dequeue A → enqueue B, C; dequeue B → enqueue D; dequeue C → D already queued; dequeue D → enqueue E; dequeue E). - The orders diverge at the third visit: DFS commits deep (D); BFS finishes A's whole level (C) first. - BFS route to E has 3 edges (A-B-D-E); DFS happened to walk further before finding it.

Stretch: Add edge **C–E** and re-run both (DFS: A, B, D, C, E unchanged — check it!; BFS: A, B, C, D, E unchanged, but E is now *discovered* via C — the conga line's tapping order changes). Then: which traversal would you pick to detect whether the graph is connected, and does it matter? (Either — both visit all reachable nodes in O(nodes + edges).)

2.3.1 Pathfinding: Dijkstra & A*

Rope Graph Dijkstra (human demo · 20 min · whole class)

Resources: 6 floor stations (paper plates) A-F taped across the room; rope/string edges with a pegged cost label on each: **A–B 4, A–C 2, B–C 1, B–D 5, C–D 8, C–E 10, D–E 2, D–F 6, E–F 3**. Each station's student holds a whiteboard showing their current best distance (start at ∞) and "via" node; red sashes (or hats) mark settled nodes; the class is the priority queue.

How it runs: 1. Scenario: A is the depot, F the delivery address; edge labels are minutes. Station A sets its board to **0**; everyone else shows ∞ . 2. Loop, chanted by the class: **"Settle the smallest unsettled!"** The unsettled station with the smallest board value puts on the red sash (settled — value now final and may never change). 3. The newly settled station tugs each of its ropes: every unsettled neighbour computes *settled value + rope cost* and updates their board **only if it's an improvement**, also recording "via (node)". 4. Repeat until F is settled. Then walk the path backwards using the "via" boards, laying a coloured ribbon along the ropes. 5. Pause moments to exploit: when C settles, B improves from 4 to 3 — "the detour beats the direct rope!"; and when D is tempted to update a settled node, the sash forbids it.

Answers / expected outcomes: - Settle order and final distances: **A = 0, C = 2, B = 3, D = 8, E = 10, F = 13**. - Update log: settle A → B: 4 (via A), C: 2 (via A). Settle C → B improves to **3** (via C), D: 10 (via C), E: 12 (via C). Settle B → D improves to **8** (via B). Settle D → E improves to **10** (via D), F: 14 (via D). Settle E → F improves to **13** (via E). Settle F — done. - Ribbon path (via-boards read back from F): **A → C → B → D → E → F**, total 2 + 1 + 5 + 2 + 3 = **13**. - Sanity checks the class can verify: A-B-D-F = 15, A-C-D-F = 16, A-B-D-E-F = 14, A-C-B-D-F = 14, A-C-E-F = 15 — all beaten by 13. - Rules students must be able to

recite: settle the smallest tentative unsettled node; relax neighbours; once settled, a distance is final; Dijkstra needs non-negative weights.

Stretch: Change rope B–D to cost 1 and predict the new answer before re-running (A–C–B–D–E–F = $2+1+1+2+3 = 9$). Then: Dijkstra found shortest paths to every settled node, not just F — read them all off the via-boards. What did that cost us? (It explored everywhere — motivation for A*.)

Sat-Nav Duel: Dijkstra vs A* (unplugged race · 20 min · two teams)

Resources: The same rope graph as *Rope Graph Dijkstra* plus one new station Z with edges A–Z 1 and Z–E 25 (Z is a tempting cheap side-street that leads nowhere useful). Printed heuristic cards — straight-line estimates to F: $h(A)=9$, $h(B)=8$, $h(C)=9$, $h(D)=4$, $h(E)=3$, $h(F)=0$, $h(Z)=20$. Two team scribes with trace tables.

How it runs: 1. Split the class: **Team Dijkstra** runs the standard algorithm (expand smallest g = distance so far). **Team A*** runs the same procedure but expands smallest $f(n) = g(n) + h(n)$, using the heuristic cards. Both hunt the shortest A → F route. 2. Both teams keep a public tally of every node they *expand* (settle). The race is won on **fewest expansions with the correct answer**. 3. Run Team Dijkstra first (or in parallel on two boards). Watch Z: with $g = 1$ it is settled **second** — Dijkstra wastes a visit on the cheap dead end. 4. Run Team A: Z's ticket is $f = 1 + 20 = 21$, so it sits *unexpanded at the back of the queue while the goal is reached at $f = 13$* . 5. Debrief with the KO table: Dijkstra = uninformed, explores outward in all directions; A = informed, **adds a heuristic estimate of remaining distance**, directed toward the goal; identical answers here because h never overestimates (**admissible**). If $h(n) = 0$ everywhere, A is* Dijkstra.

Answers / expected outcomes: - Both teams' final answer: **A → C → B → D → E → F, cost 13** (Z routes: A–Z–E–F = $1+25+3 = 29$ — never competitive). - Team Dijkstra expansions in order: **A(0), Z(1), C(2), B(3), D(8), E(10), F(13)** — 7 nodes. - Team A expansions with f -values: A ($f=9$), C ($f=2+9=11$), B ($f=3+8=11$), D ($f=8+4=12$), E ($f=10+3=13$), F ($f=13+0=13$) — 6 nodes; Z ($f=21$) is never expanded. - Verified admissibility ($h \leq$ true remaining cost to F): A $9 \leq 13$, B $8 \leq 10$, C $9 \leq 11$, D $4 \leq 5$, E $3 \leq 3$, F $0 \leq 0$, Z $20 \leq 28$. ✓ - Winner: A — same optimal path, fewer expansions, because the heuristic steers it away from Z.

Stretch: Sabotage round: change $h(D)$ to 12 (an overestimate — true remaining is 5) and discuss what *could* now go wrong (inadmissible heuristic → A may commit elsewhere and return a non-optimal path; optimality guarantee lost). Then set every h to 0 and confirm Team A's expansion order becomes exactly Team Dijkstra's.

Mark This Trace: The Fictional Student's Dijkstra (spot-and-fix-the-bug · 15 min · pairs)

Resources: Printed "student answer" below per pair (same 6-node graph as *Rope Graph Dijkstra*: A–B 4, A–C 2, B–C 1, B–D 5, C–D 8, C–E 10, D–E 2, D–F 6, E–F 3); green pens for marking; blank trace table for the correction.

The fictional student's work (find A → F):

Step	Settled	A	B	C	D	E	F
1	A (0)	0	4	2	∞	∞	∞
2	B (4)	—	4	2	9	∞	∞
3	C (2)	—	—	2	9	12	∞
4	D (9)	—	—	—	9	11	15
5	E (11)	—	—	—	—	11	14
6	F (14)	—	—	—	—	—	14

"Shortest path = $A \rightarrow B \rightarrow D \rightarrow E \rightarrow F$, cost 14."

How it runs: 1. Pairs mark the trace like an examiner: find the **first step that breaks a rule of Dijkstra**, name the rule, and explain how the error snowballs into the final answer. 2. A hint after 3 minutes if needed: "compare the tentative values available at step 2 — which node *should* have been settled?" 3. Pairs then produce the fully corrected table and path on the blank grid. 4. Debrief the meta-lesson: the student's arithmetic after step 2 is flawless — one broken rule early corrupts everything, which is why the settle-the-smallest rule must be mechanical.

Answers / expected outcomes: - **The error (step 2):** the student settled B (tentative 4) while C had tentative 2 — breaking the rule *always settle the unsettled node with the smallest tentative distance*. Consequence: B is finalised at 4, so the cheaper route $A \rightarrow C \rightarrow B$ ($2 + 1 = 3$) is never applied; D becomes 9 instead of 8, E 11 instead of 10, F 14 instead of 13. - Note for sharp students: the rest of the student's table is internally consistent given the error — every later number follows correctly from the bad step 2. - Corrected trace: settle **A(0), C(2), B(3), D(8), E(10), F(13)**; updates as in *Rope Graph Dijkstra*. - Correct answer: **$A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow F$, cost 13** (student's 14 via A-B-D-E-F is not optimal).

Stretch: Write the one-sentence examiner's comment that would have saved this student, then design a different single-error trace (e.g. one missed relaxation, or updating an already-settled node) and swap with another pair to mark.

Admissible or Not? (card sort · 10 min · small groups)

Resources: Per group: the 6-node graph diagram from *Rope Graph Dijkstra* with the **true remaining costs to F** printed beside it (A = 13, B = 10, C = 11, D = 5, E = 3, F = 0); two sorting mats labelled **ADMISSIBLE** and **NOT ADMISSIBLE**; nine heuristic cards.

Heuristic cards: ① $h(A)=13$ · ② $h(B)=12$ · ③ $h(C)=0$ · ④ $h(D)=5$ · ⑤ $h(E)=4$ · ⑥ $h(F)=0$ · ⑦ $h(F)=2$ · ⑧ "h(n) = 0 for every node" · ⑨ "h(n) = straight-line distance to F on an accurate map"

How it runs: 1. Recap the definition on the board: a heuristic is **admissible** if it **never overestimates** the true remaining cost — the condition for A* to guarantee the optimal path. 2. Groups sort the nine cards onto the two mats; every placement needs a spoken justification comparing h against the true value. 3. Whole-class check, then the two discussion bombs: card ⑧ — "if it's admissible, why don't we always use it?" and card ③ — "is a uselessly-low heuristic still admissible?"

Answers / expected outcomes: - **Admissible:** ① ($13 \leq 13$ — equal is allowed), ③ ($0 \leq 11$ — admissible but uninformative), ④ ($5 \leq 5$), ⑥ ($0 \leq 0$ — h at the goal must be 0), ⑧ (never overestimates anywhere — but A degenerates into Dijkstra: optimal, just slow), ⑨ (a straight line can never be longer than any actual route — the classic admissible heuristic). - **Not admissible:** ② ($12 > 10$), ⑤ ($4 > 3$), ⑦ ($2 > 0$ — claims the

goal is 2 away from itself). - Key synthesis: *admissible = safe (optimality guaranteed)*; closer to the true cost without exceeding it* = fewer nodes expanded. $h = 0$ is the safe-but-slow extreme; overestimates are fast-but-wrong risks.

Stretch: Groups invent the "best legal" heuristic for this graph (h equal to the true costs — perfectly informed: A* then walks straight along the optimal path) and explain why real sat-navs can't have it (knowing true remaining costs would mean the problem is already solved — so we settle for cheap estimates like straight-line distance).